

Constructor Function em .Js

#programming

#javascript

Esse é um método de usar Functions em .Js para construir Objects em .Js de forma automática.

Base para o conhecimento

Em JavaScript, **constructor functions** são funções usadas para criar objetos com **new**. Elas servem como um “molde” para gerar várias instâncias com a mesma estrutura.

Dentro da função, **this** representa o objeto que será criado.

Exemplo:

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
const a = new Person("Walter", 17);  
const b = new Person("Maria", 20);
```

- **new** cria um novo objeto
- liga esse objeto ao **Person.prototype**
- define **this** como esse novo objeto
- retorna o objeto automaticamente

Resultado:

```
a.name // "Walter" b.age // 20
```

É a base do modelo antigo de orientação a objetos em JavaScript (antes de **class**, que por baixo usa a mesma ideia).

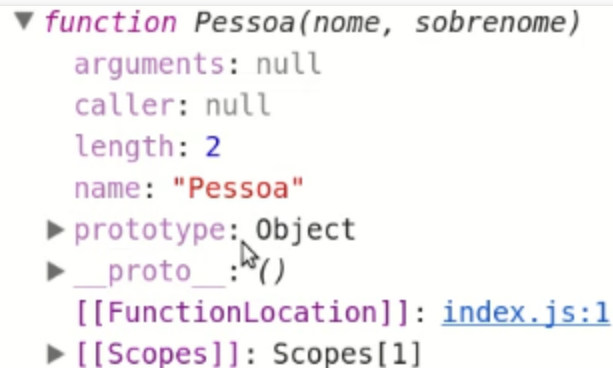
Usando **methods** em uma **constructor function**

Se atribuíssemos a uma **constructor function**, uma **method** diretamente, mesmo que ao usar o exemplo dado em Constructor Function em .Js > Base para o conhecimento, o sistema criaria **objects** de forma automática, mas todo novo **object** criado possuiria uma **function** própria, repetindo-se em grande quantidade. Um verdadeiro desperdício de memória.

Para isso, o JavaScript conta com as propriedades `prototype` e `__proto__`.

`prototype`

Essa propriedade pertence a todas as funcoes, e esta "escondida" nelas.



```
▼ function Pessoa(nome, sobrenome)
  arguments: null
  caller: null
  length: 2
  name: "Pessoa"
  ▶ prototype: Object
  ▶ __proto__: Function()
  [[FunctionLocation]]: index.js:1
  ▶ [[Scopes]]: Scopes[1]
```

Aproveitando isso, armazena-se a funcao em `function.prototype` e assim, ao chama-la em algum `object` construido com uma `constructor function`, a mesma `function` e reutilizada, liberando muito mais espaco.

Exemplo:

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}

Person.prototype.nameAndAge(){
  return this.name + ", " + this.age + " anos."
}

const a = new Person("Walter", 17);

console.log(a.nameAndAge());
```

O object `a` nao possui `nameAndAge`, mas esse `method` esta referenciada atraves de `__proto__`, que aponta para `prototype` da constructor function que a formou.

`__proto__`

Essa propriedade pertence ao `object` e aponta para o `prototype` da constructor function que a formou.